Transformations for Evolving Property Graph Schemas (Appendix)

Gauvain Devillez
University of Mons
Mons, Belgium
gauvain.devillez@umons.ac.be

Stefania Dumbrava
ENSIIE, INRIA, IRIF, SAMOVAR
Evry, France
stefania.dumbrava@ensiie.fr

Angela Bonifati
Lyon 1 University, CNRS Liris and IUF
Lyon, France
angela.bonifati@univ-lyon1.fr

PVLDB Reference Format:
Gauvain Devillez, Stefania Dumbrava, and Angela Bonifati.
Transformations for Evolving Property Graph Schemas (Appendix).
PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/GauvainD/GRAFT.

1 Technical Details

This appendix complements Section 4. Section 1.1 details meta-transformation generation and complexity. Section 1.2 establishes the size bounds of the induced meta-graph. Section 1.3 provides details concerning the formal properties of GRAFT.

1.1 Meta-Transformation Generation

Clique Detection. Algorithm 1 presents a basic version of the Bron–Kerbosch algorithm [1]. In our framework, maximal clique detection is used as a preprocessing step in Algorithm 1 to collapse permutation-equivalent edit operations into contracted edit operations, and as a postprocessing step to expand representative sequences when necessary. We construct a compatibility graph whose vertices are grounded edit operations and whose edges connect mutually compatible operations, i.e., operations e and e' such that each may follow the other and share the same predecessor context. Maximal cliques correspond to sets of edit operations that can be applied in arbitrary order.

Worst-Case Complexity. Let D denote the length of the longest path in the edit-operation automaton, Δ the maximum number of successors of any edit operation, and k the maximum number of edit operations contracted into a single contracted edit operation.

Theorem 1. *Let D denote the maximum depth of the edit-operation DAG, Δ the maximum number of successors of any edit operation, and k the maximum size of a contracted clique. Algorithm 1 runs in worst-case time $O\left(2^k \cdot \frac{(2^k\Delta)^D - 1}{2^k\Delta - 1}\right)$.*

Proof. Assume first that no edit operations are contracted and that the successor relation forms a rooted directed acyclic graph with a single starting operation r. Partition edit operations into layers $E_0, \ldots, E_D$ according to their distance from r.

In the worst case, every operation in E_i precedes every operation in E_{i+1}, with $|E_{i+1}| \leq \Delta$. An operation at depth i may then be explored via at most Δ^{i-1} distinct paths. The total number of explored states is therefore bounded by $1 + \Delta\frac{\Delta^D - 1}{\Delta - 1}$.

Consider now contraction. A contracted edit operation representing a clique of size k corresponds to 2^k subsets of underlying operations. These subsets can be enumerated in $O(2^k)$ time using incremental subset generation.

In the worst case, every operation belongs to a clique of size at most k, so each layer may contain up to $2^k\Delta$ effective successors. Substituting Δ with $2^k\Delta$ yields the stated bound. □

Algorithm 1: `Bron-Kerbosch`

Input: P: set of grounded edit operations, R: current clique (initially $\emptyset$), X: excluded operations (initially $\emptyset$)
Output: Maximal cliques of mutually compatible edit operations

1 **if** $P = \emptyset$ *and* $X = \emptyset$ **then**
2 | output R
3 **end**
4 **foreach** $e \in P$ **do**
5 | $P_e \leftarrow \{e' \in P \cap \mathrm{Next}(e) \mid \mathrm{pred}(e) = \mathrm{pred}(e')$ and $e \in \mathrm{Next}(e')\}$
6 | `Bron-Kerbosch`$(P_e, R \cup \{e\}, X \cap P_e)$
7 | $P \leftarrow P \setminus \{e\}$
8 | $X \leftarrow X \cup \{e\}$
9 **end**

Theorem 1 (Exhaustive Duplication-free Meta-transformation Derivation). *Algorithm 1 enumerates exactly the duplication-free transformation sequences that start with an edit operation in* Start *and respect the* Next *relation.*

Proof. Generation increases transformation length iteratively. Let $\mathcal{M}_t$ denote the set of generated transformations with $t \geq 1$ edit operations. The set $\mathcal{M}_1$ is obtained at line 2 and contains only transformations composed of one edit operation which is allowed by the Start rules by construction.

Assume $t > 1$ and consider the set $\mathcal{M}_t$. This set is generated after $t-1$ iterations of the loop at line 3. Let $M = \{e_t \circ e_{t-1} \circ \cdots \circ e_0\} \in \mathcal{M}_t$.

M is obtained from a transformation M_{t-1} in $\mathcal{M}_{t-1}$ by instruction at line 8. By induction on t, $e_0 \in$ Start. The loop at line 6 restricts e_t to be in $\mathrm{Next}(e_{t-1})$, therefore ensuring Next is respected. Lastly, line 7 ensures e_t is not an edit operation of M_{t-1} and thus that M does not contain duplicated edit operations.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

For each t, if $\mathcal{M}_t$ is not empty, the loop at line 3 iterates and $\mathcal{M}_t$ is added to the result. If $\mathcal{M}_t$ is empty, then for each transformation $M_{t-1} = \{e_{t-1} \circ \cdots \circ e_0\} \in \mathcal{M}_{t-1}$, either no edit operation is allowed to follow e_{t-1} by Next ($\mathsf{Next}(e_{t-1}) = \emptyset$) or all candidate edit operations are already in M_{t-1} ($\forall e \in \mathsf{Next}(e_{t-1}), e \in \{e_{t-1}, \ldots, e_0\}$). By lines 6 and 7, the algorithm will then indeed produce an empty $\mathcal{M}_t$. In this case, there is also no transformation of length greater than $t+1$ and stopping computation from the condition in line 3 will not miss any transformation.

Therefore, all the transformations are produced and they all respect the stated constraints. □

Theorem 2 (Worst-Case Time Complexity of Meta-Transformation Generation). *Let n be the number of distinct edit operations. Then Algorithm 1 runs in $O(n!)$ in the worst case. With clique compression, this becomes $O(2^n)$.*

Proof. Notice first that Algorithm 1 generates each transformation in $O(1)$. Its worst-case time complexity is thus equal to the worst-case number of transformations.

Let n denote the number of distinct edit operations. In the worst case, $|\mathsf{Start}| = n$ and, for every edit operation e, $|\mathsf{Next}(e)| = n$. Since duplicate edits are disallowed, a transformation of length t can be extended by at most $n - t$ operations.

Hence, the number of transformations of length t is bounded by the number of permutations of t elements drawn from n, i.e., $P(n,t) = \frac{n!}{(n-t)!}$. Summing over all lengths produces $\sum_{t=1}^{n} P(n,t) \in O(n! \cdot n)$. This worst case corresponds to a degenerate Next relation in which every edit operation may follow any other. When a subset of edit operations is pairwise compatible and commutative[1], we treat it as a *clique*. Such a clique can be compressed into a symbolic edit operation during enumeration. Since permutations of the same subset are equivalent under commutativity, it suffices to enumerate subsets rather than permutations, reducing the worst-case number of transformations to 2^n. □

Theorem 3 (Worst-Case Time Complexity of Transformation Generation). *Let n be the number of distinct grounded edit operations. Algorithm 2 runs in $O(2^n \cdot n)$ time in the worst case (with clique compression).*

Proof. Let n be the number of distinct grounded edit operations produced at line 2. Since meta-edit variables range only over elements of the finite input schemas, n is finite and polynomial in the input size. By Theorem 2, with clique compression, Algorithm 1 (line 3) generates at most 2^n duplication-free transformations. Each transformation has length at most n. Algorithm 2 iterates once per transformation (line 4) and applies at most n edit operations per transformation (line 6), each in polynomial time.

Therefore, the overall worst-case running time is $O(2^n \cdot n)$, which is exponential in the number of grounded edit operations. □

1.2 Meta-Graph Schema Search Space

Theorem 4 (Meta-Graph Size Bound). *Let n be the total number of distinct edit operations generated by the meta-transformations in $\mathcal{M}^*$. Then, the number of distinct schemas reachable under a finite set of meta-transformations is bound by 2^n. Consequently, the meta-graph contains at most 2^n vertices and at most $|\mathcal{M}^*| \cdot 2^n$ edges.*

Proof. Let S be a schema in the meta-graph and let n be the total number of distinct edit operations generated by the meta-transformations. This schema is either one of the input schemas or obtained from one of them using a transformation. These transformations, by construction, cannot insert elements that are not present in one of the input schemas.

Let E be the set of all the n edit operations produced by the meta-transformations, and let S_i be a schema in the meta-graph. There is a sequence of unique edit operations that transform the source S into S_i. Indeed, consider the shortest path in the metagraph from S to S_i and let M be the composition of these transformations. M is a transformation too since it is a composition of edit operations. It contains at most n distinct edit operation and transforms S into S_i.

Thus, the total number of schemas in the metagraph is bounded by the total number of sequences of edit operation, which is $O(n!)$. However, every permutation of the same edit operations leads either to the same schema (the sets of objects added and removed are the same) or is an invalid sequence (e.g., setting a node property before adding it). Therefore, the number of schemas in the metagraph is bounded by the number of sets of edit operations, that is $O(2^n)$. A given transformation applied to a schema can only produce one schema. The meta-graph can thus have at most 2^n edges per meta-transformation, that is, at most $|\mathcal{M}^*| \cdot 2^n$. □

1.3 GRAFT Theoretical Properties

Theorem 4 (GRAFT Termination). *Let $\mathcal{S}$ be a finite set of schemas and $\mathcal{M}^*$ a finite set of meta-transformations whose variables range over elements of $\mathcal{S} \cup \{T\}$. Then Algorithm 3 terminates.*

Proof. We prove termination by showing that only finitely many distinct schemas can be generated.

By Theorem 2, grounding the meta-transformations over the input schemas yields a finite set $\mathcal{M}$ of duplication-free transformations. By construction, transformations do not contain duplicate edit operations (Theorem 1), and each edit operation refers only to schema elements occurring in the input schemas. Hence, no transformation can introduce fresh symbols.

Let T be the finite set of schema elements (nodes, edges, labels, properties) occurring in the input schemas. Every schema generated during the execution of the algorithm is obtained by applying a finite sequence of grounded transformations to the source schema. Since transformations cannot introduce elements outside T, every generated schema is a subset of T. Therefore, by Theorem 4, the number of distinct reachable schemas is finite.

The algorithm maintains a set G containing all previously generated schemas. No schema is ever removed from G, and a schema is added to $\mathcal{S}_{NEW}$ only if it is not already contained in G. Hence, each schema can be inserted into $\mathcal{S}_{NEW}$ at most once.

Since the total number of distinct reachable schemas is finite, $\mathcal{S}_{NEW}$ can grow only finitely many times. Once all reachable schemas have been added to G, no new schema can be generated, and $\mathcal{S}_{NEW}$ strictly decreases at each iteration. Therefore, the while-loop at line 8 terminates.

[1] *e.g.*, adding multiple properties to a node.

All remaining loops iterate over finite sets ($\mathcal{M}^*$ and $\mathcal{S}'$), and thus also terminate. Hence, the algorithm terminates. □

Theorem 5 (GRAFT Time Complexity). *Let d denote the minimum number of transformation steps required to map S to T, b the branching factor induced by grounded meta-transformations, and C the cost of executing Algorithm 2. Without pruning, Algorithm 3 explores $O(b^d)$ schemas in the worst case, with runtime $O(b^d \cdot C)$. With pruning and width k, at most k^i schemas are retained at depth i, leading to the runtime bound $O(k^d \cdot b \cdot (C + \log k))$.*

Proof. We model the execution of Algorithm 3 as a search tree of depth d, where each level corresponds to one transformation.

Without pruning, at each step, applying the grounded meta-transformations to a schema produces at most b successor schemas. In the worst case, the full search tree up to depth d is explored, generating $\sum_{i=1}^{d} b^i = O(b^d)$ distinct schemas. For each generated schema, Algorithm 2 is executed once, at cost C. Hence, the overall runtime is $O(b^d \cdot C)$.

When pruning is enabled, at depth i the algorithm retains at most k^i schemas. Expanding these schemas yields at most $k^i \cdot b$ candidates. For each candidate, Algorithm 2 is run at cost C, while keeping the top-k schemas by similarity to T. This selection can be implemented using a size-bounded heap, with $O(\log k)$ update cost per candidate. Summing over all depths gives a total cost of $\sum_{i=0}^{d-1} k^i \cdot b \cdot (C + \log k)$ and bound $O(k^d \cdot b \cdot (C + \log k))$. □

References

[1] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (1973), 575–577.